

1. Zadatak - Implementacija vlastite konvolucijske neuronske mreže (MNIST)

U ovom zadatku implementirat ćete konvolucijsku neuronsku mrežu (CNN) za klasifikaciju znamenki iz MNIST dataseta.

Arhitektura mreže prikazana je na slici student_CNN.png.

Vaš zadatak je implementirati tu arhitekturu točno prema prikazanoj strukturi.

1) Implementacija mreže (model_student.py)

U datoteci model_student.py nalazi se template klase StudentCNN.

Potrebno je:

- definirati konvolucijski dio mreže (Conv → ReLU → MaxPool)
- izračunati točan broj ulaznih značajki za prvi Linear sloj
- definirati potpuno povezani (fully connected) dio
- implementirati forward metodu

Mreža mora odgovarati arhitekturi sa slike.

Izlazni tensor mora imati oblik:

[B, 10]

gdje je:

- B - veličina batcha
- 10 - broj klasa (znamenke 0-9)

2) Konstrukcija modela (train.py)

U skripti train.py nalazi se infrastruktura za:

- učitavanje MNIST dataseta
- podjelu na train i validacijski skup (90% / 10%)
- DataLoader-e
- trening petlju
- validacijsku petlju
- logiranje na TensorBoard

U označenom TODO dijelu potrebno je:

- uvesti klasu StudentCNN
- konstruirati model s odgovarajućim parametrima
- prebaciti model na odgovarajući uređaj (CPU/GPU)

3) Što radi train.py skripta?

Skripta:

1. Loada MNIST dataset
2. Dijeli podatke na train i validation skup
3. Kreira DataLoader-e
4. Instancira model
5. Trenira model kroz zadani broj epoha
6. U trening petlji logira **loss**
7. U validacijskoj petlji logira **accuracy**

Cilj je da model postigne visoku validacijsku točnost ($\geq 97\%$).

Važno

- Ne mijenjati strukturu train.py skripte.
- Implementacija mreže mora biti u model_student.py.

2. Zadatak - Implementacija konvolucijske neuronske mreže (CIFAR-10)

U ovom zadatku implementirat ćete konvolucijsku neuronsku mrežu (CNN) za klasifikaciju slika iz CIFAR-10 dataseta.

Arhitektura mreže prikazana je na slici CIFAR_CNN.png.

Vaš zadatak je implementirati tu arhitekturu točno prema prikazanoj strukturi.

1) Implementacija mreže (model_cifar.py)

U datoteci model_cifar.py nalazi se template klase CifarCNN.

Potrebno je:

- definirati konvolucijski dio mreže (Conv → ReLU → MaxPool)
- pravilno pratiti promjenu dimenzija kroz tri konvolucijska bloka
- izračunati točan broj ulaznih značajki za prvi Linear sloj
- definirati potpuno povezani (fully connected) dio
- implementirati forward metodu

Mreža mora odgovarati arhitekturi sa slike.

Ulazni tensor ima oblik: [B, 3, 32, 32]

Nakon trećeg pooling sloja izlaz mora biti: [B, 128, 4, 4]

Flatten dimenzija mora iznositi: 2048

Izlazni tensor mora imati oblik: [B, 10]

gdje je:

- B – veličina batcha
- 10 – broj klasa (CIFAR-10)

2) Konstrukcija modela (train_cifar.py)

U skripti train_cifar.py nalazi se infrastruktura za:

- učitavanje CIFAR-10 dataseta
- podjelu na train i validacijski skup (90% / 10%)

- DataLoader-e
- trening petlju
- validacijsku petlju
- logiranje na TensorBoard

U označenom TODO dijelu potrebno je:

- uvesti klasu CifarCNN
- konstruirati model s odgovarajućim parametrima
- prebaciti model na odgovarajući uređaj (CPU/GPU)

3) Što radi train_cifar.py skripta?

Skripta:

- Loada CIFAR-10 dataset
- Dijeli podatke na train i validation skup
- Kreira DataLoader-e
- Instancira model
- Trenira model kroz zadani broj epoha
- U trening petlji logira loss
- U validacijskoj petlji logira accuracy

Cilj je da model postigne stabilnu validacijsku točnost na CIFAR-10 datasetu.

Važno

Ne mijenjati strukturu train_cifar.py skripte.

Implementacija mreže mora biti u model_cifar.py.